Probeklausur: Echtzeitsysteme (EZS)

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung & Erwartungshorizont

Allgemeine Hinweise

- **Prüfung:** Semesterabschlussprüfung
- **Bearbeitungszeit:** 90 Minuten
- **Hilfsmittel:** Keine (Closed Book)
- **Maximale Punktzahl:** 100 Punkte

Aufgabe 1: Scheduling-Theorie & RMS (25 Punkte)

Ein neues Steuergerät soll drei periodische Tasks im präemptiven Multitasking verarbeiten. Für die Tasks wurden folgende Parameter (Worst-Case Execution Time C_i, Periode T_i) ermittelt. Es gelten implizite Deadlines ($D_i = T_i$).

- τ_1: $C_1 = 2\,\text{ms}$, $T_1 = 8\,\text{ms}$
- τ_2: $C_2 = 3\,\text{ms}$, $T_2 = 12\,\text{ms}$
- τ_3: $C_3 = 4\,\text{ms}$, $T_3 = 24\,\text{ms}$

1.1 Berechnen Sie die Gesamt-CPU-Auslastung U des Systems. Zeigen Sie Ihren Rechenweg. *(5 Punkte)*

1.2 Vergeben Sie die Prioritäten der Tasks nach den Regeln des *Rate Monotonic Scheduling (RMS)*. Begründen Sie Ihre Wahl in einem Satz. *(5 Punkte)*

1.3 Prüfen Sie mithilfe der Liu-und-Layland-Schranke, ob dieses Task-Set garantiert schedulbar ist. Die Schranke für $n = 3$ lautet $\approx 78{,}0\,\%$. *(5 Punkte)*

1.4 Nennen Sie zwei der strikten mathematischen Annahmen, die Liu und Layland 1973 für diesen Beweis trafen, welche in der Realität jedoch fast nie zu $100\,\%$ erfüllt sind. *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

1.1 Auslastung U:

$$U = \sum_{i=1}^{3} \frac{C_i}{T_i} = \frac{2}{8} + \frac{3}{12} + \frac{4}{24} = \frac{1}{4} + \frac{1}{4} + \frac{1}{6} = \frac{3}{12} + \frac{3}{12} + \frac{2}{12} = \frac{8}{12} = \frac{2}{3} \approx 0{,}667$$

Die Gesamt-CPU-Auslastung beträgt $U = \frac{2}{3}$ (ca. **66,7 %**). *(5 Punkte)*

1.2 RMS-Prioritäten: Regel: Je kürzer die Periode, desto höher die Priorität (*Rate Monotonic*).

- **Höchste Priorität:** τ_1 ($T_1 = 8\,\text{ms}$)
- **Mittlere Priorität:** τ_2 ($T_2 = 12\,\text{ms}$)
- **Niedrigste Priorität:** τ_3 ($T_3 = 24\,\text{ms}$)

(5 Punkte)

1.3 RMS-Schrankentest: Da die berechnete Systemauslastung $U = \frac{2}{3} \approx 66{,}7\,\%$ strikt kleiner ist als die Liu-und-Layland-Schranke von $\approx 78{,}0\,\%$ für $n = 3$ Tasks, ist das System **garantiert (hinreichend) schedulbar**. Keine Task wird ihre Deadline verpassen. *(5 Punkte)*

1.4 Liu-&-Layland-Annahmen (zwei der folgenden reichen für volle Punktzahl; laut Vorlesung):

1. Alle Tasks sind strikt periodisch (keine sporadischen Interrupts).
2. Deadline entspricht exakt der Periode ($D_i = T_i$).
3. Tasks sind völlig unabhängig (keine Mutexe, kein Shared Memory).
4. Die Laufzeit des Schedulers / der Context-Switch-Overhead ist exakt 0.

(10 Punkte)

Aufgabe 2: Synchronisation & Priority Inversion (20 Punkte)

2.1 Erklären Sie den fundamentalen konzeptionellen Unterschied zwischen einem CMSIS-Mutex und einem binären Semaphor in FreeRTOS. Gehen Sie dabei auf die Begriffe *Ownership* und *Verwendung in Interrupts (ISR)* ein. *(10 Punkte)*

2.2 Beschreiben Sie das Phänomen der *Priority Inversion* (bekannt vom Mars Pathfinder). Welche drei Task-Prioritäten sind daran beteiligt und wie löst das *Priority Inheritance Protocol (PIP)* dieses Problem? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Zu 2.1 (Mutex vs. Semaphor): Ein Mutex besitzt **Ownership** (Besitzrecht). Nur die Task, die den Mutex gesperrt (`Acquire`) hat, darf ihn auch wieder entsperren (`Release`). Er dient primär dem Schutz gemeinsamer Daten (Critical Sections). Wegen des Ownerships darf ein Mutex **niemals** in einer Hardware-ISR verwendet werden.

Ein binärer Semaphor hat kein Ownership und dient der Signalisierung. Eine ISR darf ihn freigeben (`Release`), um eine wartende Task aufzuwecken (Deferred Interrupt Processing). *(10 Punkte)*

Zu 2.2 (Priority Inversion & PIP): Beteiligt sind eine Low-, eine Medium- und eine High-Prio-Task. Die Low-Task sperrt einen Mutex. Die High-Task möchte denselben Mutex und blockiert. Nun wird die Medium-Task ready, präemptiert die Low-Task (da $P_{\text{Med}} > P_{\text{Low}}$) und rechnet weiter. Die High-Task wartet damit indirekt auf die Medium-Task — eine Prioritätsinversion liegt vor.

Lösung durch PIP: Das Betriebssystem erkennt die Blockade und *vererbt* der Low-Task temporär die Priorität der High-Task (Priority Boost). Die Medium-Task kann die Low-Task nicht mehr präemptieren. Die Low-Task beendet ihre Critical Section zügig, gibt den Mutex frei und fällt auf ihre alte Priorität zurück. Die High-Task übernimmt den Mutex. *(10 Punkte)*

Aufgabe 3: Multicore & Architektur (20 Punkte)

Sie entwerfen ein System auf dem STM32H755 (Cortex-M7 und Cortex-M4).

3.1 Erklären Sie den Unterschied zwischen SMP (*Symmetric Multiprocessing*) und AMP (*Asymmetric Multiprocessing*). Warum wird für harte Echtzeitsysteme nahezu ausschließlich AMP verwendet? *(10 Punkte)*

3.2 Was ist ein *Cache Coherency Failure*? Beschreiben Sie, wie dieser Fehler auftritt, wenn der M7-Kern über Write-Back-Caches in den geteilten SRAM4 schreibt und der M4-Kern versucht, diese Daten zu lesen. Wie löst man dieses Problem hardwareseitig? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Zu 3.1 (SMP vs. AMP): Bei SMP sind alle Kerne (typischerweise) identisch und werden von einem einzigen OS verwaltet, das Tasks dynamisch zwischen den Kernen verschiebt (Load Balancing). Bei AMP sind die Kerne oft unterschiedlich (hier M7/M4) und führen strikt getrennte Programme bzw. Betriebssysteme aus.

Für harte Echtzeit wird AMP genutzt, weil SMP durch Task-Migrationen, Cache-Thrashing und unvorhersehbare Bus-Konflikte die Worst-Case Execution Time (WCET) unberechenbar macht (mangelnder Determinismus). AMP bietet physikalische Isolation der Zeitdomänen. *(10 Punkte)*

Zu 3.2 (Cache-Kohärenz): Beim Write-Back-Verfahren speichert der M7 geschriebene Variablen zunächst in seinem internen L1-Cache. Der physikalische SRAM4 wird nicht sofort aktualisiert (Daten dort sind *stale*). Der cache-lose M4 liest dann veraltete Werte aus dem SRAM4 — das System asynchronisiert sich.

Lösung: Über die Memory Protection Unit (MPU) des M7 wird der SRAM4-Adressbereich als *Non-Cacheable* konfiguriert. Schreibzugriffe gehen dann direkt über den Bus in den RAM. *(10 Punkte)*

Aufgabe 4: Safety & Watchdogs (15 Punkte)

4.1 Der STM32 besitzt einen IWDG (*Independent*) und einen WWDG (*Window*) Watchdog. Warum reicht der IWDG nicht aus, um Code vor fehlerhaft übersprungenen Validierungs-Routinen (z. B. durch Bit-Flips) zu schützen? Wie löst das Fenster-Konzept des WWDG dieses Problem? *(8 Punkte)*

4.2 Was ist der EWI (*Early Wakeup Interrupt*)? Nennen Sie eine Aktion, die im EWI-Callback zwingend durchgeführt werden sollte, und eine, die strengstens verboten ist. *(7 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Zu 4.1: Der IWDG prüft nur, ob der Refresh *vor* Ablauf der Deadline kommt. Überspringt die CPU durch einen Softwarefehler (z. B. Bit-Flip / Pointer-Bug) Code und füttert viel zu früh, merkt der IWDG das nicht.

Der WWDG hat eine *Early Phase*: Füttert man zu früh, löst die Hardware sofort einen Reset aus. Die Software muss exakt im erlaubten zeitlichen *Allowed Window* refreshen — das erzwingt eine präzise Ausführungstaktung. *(8 Punkte)*

Zu 4.2: Der EWI (`WWDG_IRQn`) ist ein maskierbarer NVIC-Interrupt, der bei Zählerstand `0x40` — Mikrosekunden vor dem Hardware-Reset des WWDG — feuert. Typisch wird er mit höchster NVIC-Priorität (Prio 0) konfiguriert; er ist **kein** ARM-NMI.

- **Erlaubt / zwingend:** Das System in einen sicheren Zustand überführen (*Safe Shutdown*), z. B. Motor-PWMs auf 0 setzen oder einen Fehlercode loggen.
- **Verboten:** Den Watchdog im Callback füttern/refreshen — das System ist korrupt, der Reset darf nicht verhindert werden.

(7 Punkte)

Aufgabe 5: Code-Review (20 Punkte)

Der folgende C-Code soll einen zeitkritischen Hardware-Interrupt auf dem STM32 in FreeRTOS verarbeiten. Das System stürzt jedoch regelmäßig ab oder friert ein.

Finden und benennen Sie **vier** fatale Architektur- oder Programmierfehler in diesem Code-Snippet und erklären Sie kurz, warum diese in einem RTOS tödlich sind. *(Hinweis: Im Snippet sind fünf Fehler angelegt; für volle Punktzahl reichen vier korrekt benannte und begründete Fehler.)*

```c
uint8_t data_ready = 0;
osMutexId_t dataMutex;

/* Hardware Interrupt Service Routine (Top-Half) */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    osMutexAcquire(dataMutex, osWaitForever);
    data_ready = 1;
    osMutexRelease(dataMutex);
}

/* RTOS Task (Bottom-Half) */
void ProcessTask(void *argument) {
    for (;;) {
        while (data_ready == 0) {
            /* Warten auf neue Daten */
        }

        data_ready = 0;
        Filter_Complex_Data(); /* dauert ca. 10 ms */

        HAL_Delay(50);
    }
}
```

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 5

Es müssen **vier der folgenden fünf** Fehler gefunden und begründet werden (je 5 Punkte, max. 20):

1. **Mutex in der ISR:** `osMutexAcquire` / `osMutexRelease` werden in einem Hardware-Interrupt aufgerufen. Mutexe haben Ownership und dürfen niemals in ISRs verwendet werden. Zusätzlich ist ein potenziell blockierender Aufruf in der ISR architektonisch unzulässig.
2. **`osWaitForever` in der ISR:** Eine ISR darf niemals warten (`timeout > 0` bzw. `osWaitForever`). Die CPU würde mit einem Hard Fault abstürzen, da eine ISR nicht in den RTOS-*Blocked*-Zustand überführt werden kann.

3. **Polling / Busy-Wait (Spinlock) in der Task:** Die Schleife `while (data_ready == 0)` verbraucht 100 % CPU-Zeit im *Running*-Zustand. Richtig wäre es, die Task über einen (binären) Semaphor in den *Blocked*-Zustand (0 % CPU) zu versetzen und aus der ISR nur `osSemaphoreRelease` zu signalisieren.
4. **`HAL_Delay(50)` statt `osDelay(50)`:** `HAL_Delay` ist eine Bare-Metal-Funktion, die die CPU in einer aktiven Warteschleife hält. Die Task blockiert nicht für den Scheduler — das zerstört Echtzeit und Multitasking.
5. **Fehlendes `volatile`:** Die globale Variable `data_ready` wird asynchron von ISR und Task genutzt, besitzt aber kein `volatile`. Der Compiler kann die Abfrageschleife in der Task zu einer Endlosschleife wegoptimieren.

Hinweis (kein Fehler): `Filter_Complex_Data()` in der Task ist als Bottom-Half-Arbeit grundsätzlich korrekt — schwere Verarbeitung gehört in den Task-Kontext, nicht in die ISR.

(20 Punkte)